

Answer the Questions

Question 1: Why does defining a custom constructor suppress the default constructor in C#?

- Because in C#, **the compiler only generates a default parameterless constructor if you don't define any constructor yourself.**

Question 2: How does method overloading improve code readability and reusability?

- Method overloading improves **readability and reusability** by allowing multiple methods to have the **same name** while accepting different parameters.

Question 3: What is the purpose of constructor chaining in inheritance?

- The purpose of **constructor chaining in inheritance** is to make sure that the **base class is properly initialized before the derived class.**

Question 4: How does new differ from override in method overriding?

- The difference is that **override replaces the inherited virtual method**, while **new hides it.**

`override` = **replace the inherited behavior**

`new` = **hide the inherited behavior**

Question 5: Why is ToString() often overridden in custom classes?

- `ToString()` is often overridden in custom classes because the default implementation usually **doesn't give useful information about the object's data.**

Question 6: Why can't you create an instance of an interface directly?

- Because an **interface only defines a contract**, not the actual implementation of an object.

Question 7: What are the benefits of default implementations in interfaces introduced in C# 8.0?

- The main benefit of **default interface implementations**, introduced in **C# 8.0**, is that an interface can provide a **default implementation of a method.**

Benefits

- **Backward compatibility:** You can add a new method to an existing interface without immediately forcing every implementing class to implement it.
- **Code reuse:** Common behavior can be written once inside the interface.
- **Easier interface evolution:** Interfaces can be extended without breaking all existing implementations.
- **Optional customization:** A class can still provide its own implementation if it needs different behavior.

Question 8: How does C# overcome the limitation of single inheritance with interfaces?

- C# overcomes the limitation of single class inheritance by allowing a class to implement multiple interfaces, giving it multiple sets of behaviors/contracts while still inheriting from only one base class.

Question 9: What is the difference between a virtual method and an abstract method in C#?

- The main difference is that a **virtual method has an implementation**, while an **abstract method only defines what must be implemented**.

virtual	abstract
Has an implementation	Has an implementation
Override is optional	Override is required
Can exist in a normal class	Must be inside an abstract class
Base class can be instantiated	Abstract class cannot be instantiated

virtual = "Here is a default behavior; change it if you want."

abstract = "I don't have a behavior; you must provide one."